Task 4 - Interactive Feature Plan & Implementation Report

1. Feature Plan

Feature Description

A competitive two-player dice game where players take turns rolling to reach a target score of **20 points**.

Target Audience

Casual gamers looking for a simple and interactive experience.

UI Layout Sketch (Description)

- **Top Section:** Scoreboard displaying Player 1 and Player 2 scores.
- **Center Section:** Large dice image showing the current roll.
- **Bottom Section:** Two buttons side-by-side:
  - "Roll Dice"
  - "Pass Turn"
- **Overlay (Conditional):**
  - "Game Over" message
  - "Play Again" button (appears only when someone wins)

2. Button List & State Model

Button List

- **Roll Button**
  - Generates a random number (1-6)
  - Adds it to the active player's temporary score
- **Pass Button**
  - Saves the temporary score to the active player's total
  - Switches the turn to the next player
- **Reset Button**
  - Appears only when the game ends
  - Clears all scores and restarts the game

3. State Table

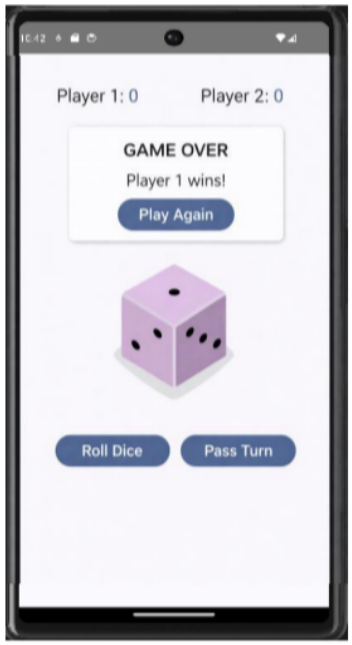
Current State	Trigger	Logic / Action	Next State	UI Feedback
Idle / Start	App Launch	Initialize score1=0, score2=0, active=P1	Player 1 Turn	Displays "Player 1's Turn" and score 0
Player 1 Turn	Click "Roll"	score1 += (1..6).random() (if total < 20)	Player 1 Turn	Dice updates; P1 score increases
Player 1 Turn	Click "Pass"	Toggle activePlayer to 2	Player 2 Turn	UI highlights Player 2
Player 2 Turn	Click "Roll"	score2 += (1..6).random() (if total < 20)	Player 2 Turn	Dice updates; P2 score increases
Player 1 or 2	Click "Roll" (total >= 20)	Set isGameOver = true	Game Over	"Roll" disabled; Winner text appears
Game Over	Click "Reset"	Reset all scores; activePlayer = 1	Player 1 Turn	Scores cleared; Roll re-enabled

4. Test Plan

1. **Sequence:** Roll (4) -> Pass
   Expected: Player 1 total = 4; turn switches to Player 2.

2. **Sequence:** Roll until total reaches 20
   Expected: "Roll" button becomes disabled; "Winner" message appears.
3. **Sequence:** Click "Reset" after win
   Expected: All scores return to 0; turn resets to Player 1.

5. Prototype / Pseudo-code




```
// State variables
var player1Score by remember { mutableStateOf(0) }
var player2Score by remember { mutableStateOf(0) }
var activePlayer by remember { mutableStateOf(1) }
var isGameOver by remember { mutableStateOf(false) }

// Scoreboard
Text("Player 1: $player1Score | Player 2: $player2Score")

// Roll Button
Button(
    onClick = {
        val roll = (1..6).random()
        if (activePlayer == 1) player1Score += roll
        else player2Score += roll

        if (player1Score >= 20 || player2Score >= 20) {
            isGameOver = true
        }
    },
    enabled = !isGameOver
) {
    Text("Roll Dice")
}

// Game Over Section
if (isGameOver) {
```

```
    Button(onClick = {
        player1Score = 0
        player2Score = 0
        activePlayer = 1
        isGameOver = false
    }) {
        Text("Play Again")
    }
}
```

6. Reflection

The primary challenge in Jetpack Compose will be managing the Turn State correctly so that the active player's score updates without unintended side effects.

To handle edge cases such as rapid repeated clicks:

- Implement debounce logic, or
- Temporarily disable buttons during animations.

To maintain state across configuration changes:

- Use `rememberSaveable` to preserve game progress during orientation changes.

This ensures reliability, smoother gameplay, and a better user experience.